



Module Name: Register File (4r, 2w)

Design Engineer(s): Suneet Tumma (.epic99 / stumma8), Vincent Pham (powlectro / vpham353)

Verification Engineer(s): Kevin Luo (ElmoTheFrog / kluo78)

Branch Name: register_file

Test Bench Folder Name: Tests/uvm/tb_register_file

Design File(s): register_file.sv, multiplexer.sv, registers.sv, wr_en_mux_arr.sv

Link to Design Spec:

<https://docs.google.com/document/d/13YPzj9ko63VZFyc2anqbJMxUwvZ5aDRPKHObSKx2CuA/edit?tab=t.0>

Overview

This module implements a register file, a small and high-speed storage block that holds general-purpose registers for the processor. There are 4 read ports (R) and 2 write ports (W), which allow simultaneous read and write operations. Writes occur on the rising edge of the clock, while reads are asynchronous and immediately reflect changes in registers. Active-low resets store the value 0 into every register synchronously. Assume input address and input values are always valid. In the case they aren't, the behavior will be unpredictable. Register 0 represents a hardwired zero register, consistent with the architectural design. In cases of simultaneous writes to the same register, Write Port 0 takes priority over Write Port 1.

I/O

- ↵ **RESET** — Active-Low Synchronous Reset
- ↵ **CLK** — Clock
- **W0** — Write Port 0
 - ↵ **we0** — Active-High Write Enable 0
 - ↵ **waddr0** — Write Address 0
 - ↵ **wdata0** — Write Data 0
- **W1** — Write Port 1
 - ↵ **we1** — Active-High Write Enable 1
 - ↵ **waddr1** — Write Address 1
 - ↵ **wdata1** — Write Data 1
- **R0** — Read Port 0
 - ↵ **raddr0** — Read Address 0
 - ⇒ **rdata0** — Read Data 0
- **R1** — Read Port 1

- ↔ **raddr1** — Read Address 1
- ⇒ **rdata1** — Read Data 1
- **R2** — Read Port 2
 - ↔ **raddr2** — Read Address 2
 - ⇒ **rdata2** — Read Data 2
- **R3** — Read Port 3
 - ↔ **raddr3** — Read Address 3
 - ⇒ **rdata3** — Read Data 3
- Placeholders for Constrained-Random Testing: 0xDEADBEEF, 0xCAFEBAFE, 0xC0FFEE01, 0x8BADF00D

Initial States

There are certain relevant conditions for the register file that are referenced by the test plan as follows:

State	Description	Register Values
RESET STATE	Represents the register file after a Reset assertion All registers store 0x00000000	Reg[x] = 0x00000000 for all x in [0..N] Example Reg[0] = 0x00000000 Reg[1] = 0x00000000 Reg[2] = 0x00000000 Reg[3] = 0x00000000 ... Reg[N] = 0x00000000
INITIALIZED STATE	Verification-only pre-load pattern used to quickly identify register contents to simplify expected-value checks during testing All registers, except Register 0, follow a simple pattern in this state	Reg[0] = 0x00000000 Reg[x] = 0x10000000 + x for all x in [1..N] Example Reg[0] = 0x00000000 Reg[1] = 0x10000001 Reg[2] = 0x10000002 Reg[3] = 0x10000003 ... Reg[N] = 0x10000000 + N

Tests

Functionality 1: Basic Read/Write Functionality in All Registers

Test Case	Description	Input Sequence	Expected Output
1.1	Read newly written data	W0 ↔ we0 = 1	R0 ⇒ rdata0 = 0xCAFEBAFE

	on the same cycle	$\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xCAFEBADE W1 $\Leftarrow$ we1 = 1 $\Leftarrow$ waddr1 = 7 $\Leftarrow$ wdata1 = 0xDEADBEEF R0 $\Leftarrow$ raddr0 = 5 R1 $\Leftarrow$ raddr1 = 7 R2 $\Leftarrow$ raddr2 = 5 R3 $\Leftarrow$ raddr3 = 7	R1 $\Rightarrow$ rdata1 = 0xDEADBEEF R2 $\Rightarrow$ rdata2 = 0xCAFEBADE R3 $\Rightarrow$ rdata3 = 0xDEADBEEF
1.2	Read data from registers not actively being written to	Begin in INITIALIZED STATE Cycle 0 W0 $\Leftarrow$ we0 = 0 $\Leftarrow$ waddr0 = X $\Leftarrow$ wdata0 = X W1 $\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X R0 $\Leftarrow$ raddr0 = 3 R1 $\Leftarrow$ raddr1 = 9 R2 $\Leftarrow$ raddr2 = 12 R3 $\Leftarrow$ raddr3 = 18 Cycle 1 W0 $\Leftarrow$ we0 = 1 $\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xCAFEBADE W1 $\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X	Cycle 0 R0 $\Rightarrow$ rdata0 = 0x10000003 R1 $\Rightarrow$ rdata1 = 0x10000009 R2 $\Rightarrow$ rdata2 = 0x1000000C R3 $\Rightarrow$ rdata3 = 0x10000012 Cycle 1 R0 $\Rightarrow$ rdata0 = 0xCAFEBADE R1 $\Rightarrow$ rdata1 = 0xCAFEBADE R2 $\Rightarrow$ rdata2 = 0x1000000C R3 $\Rightarrow$ rdata3 = 0x10000012

		R0 ⇨ raddr0 = 5 R1 ⇨ raddr1 = 5 R2 ⇨ raddr2 = 12 R3 ⇨ raddr3 = 18	
1.3	Write data and read it on the next cycle	Begin in INITIALIZED STATE Cycle 0 W0 ⇨ we0 = 1 ⇨ waddr0 = 5 ⇨ wdata0 = 0xCAFEBADE W1 ⇨ we1 = 1 ⇨ waddr1 = 12 ⇨ wdata1 = 0xDEADBEEF R0 ⇨ raddr0 = 5 R1 ⇨ raddr1 = 12 R2 ⇨ raddr2 = 3 R3 ⇨ raddr3 = 9 Cycle 1 W0 ⇨ we0 = 0 ⇨ waddr0 = X ⇨ wdata0 = X W1 ⇨ we1 = 0 ⇨ waddr1 = X ⇨ wdata1 = X R0 ⇨ raddr0 = 5 R1 ⇨ raddr1 = 12 R2 ⇨ raddr2 = 3 R3	Cycle 0 R0 ⇨ rdata0 = 0xCAFEBADE R1 ⇨ rdata1 = 0xDEADBEEF R2 ⇨ rdata2 = 0x10000003 R3 ⇨ rdata3 = 0x10000009 Cycle 1 R0 ⇨ rdata0 = 0xCAFEBADE R1 ⇨ rdata1 = 0xDEADBEEF R2 ⇨ rdata2 = 0x10000003 R3 ⇨ rdata3 = 0x10000009

		↔ raddr3 = 9	
1.4	Single write and single read for every register	Begin in INITIALIZED STATE Cycle 0 W0 ⇒ we0 = 1 ⇒ waddr0 = 0 ⇒ wdata0 = 0xCAFEBADE W1 ⇒ we1 = 0 ⇒ waddr1 = X ⇒ wdata1 = X R0 ⇒ raddr0 = 0 R1 ⇒ raddr1 = X R2 ⇒ raddr2 = X R3 ⇒ raddr3 = X Cycle 1 W0 ⇒ we0 = 1 ⇒ waddr0 = 1 ⇒ wdata0 = 0xDEADBEEF W1 ⇒ we1 = 0 ⇒ waddr1 = X ⇒ wdata1 = X R0 ⇒ raddr0 = 1 R1 ⇒ raddr1 = X R2 ⇒ raddr2 = X R3 ⇒ raddr3 = X Cycle 2 W0 ⇒ we0 = 1	Cycle 0 R0 ⇒ rdata0 = 0x00000000 R1 ⇒ rdata1 = X R2 ⇒ rdata2 = X R3 ⇒ rdata3 = X Cycle 1 R0 ⇒ rdata0 = 0xDEADBEEF R1 ⇒ rdata1 = X R2 ⇒ rdata2 = X R3 ⇒ rdata3 = X Cycle 2 R0 ⇒ rdata0 = 0xC0FFEE01

		⇒ waddr0 = 2 ⇒ wdata0 = 0xC0FFEE01 W1 ⇒ we1 = 0 ⇒ waddr1 = X ⇒ wdata1 = X R0 ⇒ raddr0 = 2 R1 ⇒ raddr1 = X R2 ⇒ raddr2 = X R3 ⇒ raddr3 = X ... Cycle N W0 ⇒ we0 = 1 ⇒ waddr0 = N ⇒ wdata0 = 0x8BADF00D W1 ⇒ we1 = 0 ⇒ waddr1 = X ⇒ wdata1 = X R0 ⇒ raddr0 = N R1 ⇒ raddr1 = X R2 ⇒ raddr2 = X R3 ⇒ raddr3 = X	R1 ⇒ rdata1 = X R2 ⇒ rdata2 = X R3 ⇒ rdata3 = X ... Cycle N R0 ⇒ rdata0 = 0x8BADF00D R1 ⇒ rdata1 = X R2 ⇒ rdata2 = X R3 ⇒ rdata3 = X
--	--	--	--

Functionality 2: Basic Write-Enable and Reset Functionality

Test Case	Description	Input Sequence	Expected Output
2.1	Unable to write when write enable is low	Begin in INITIALIZED STATE Cycle 0 W0	Cycle 0 R0

		$\Leftarrow$ we0 = 0 $\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xDEADBEEF W1 $\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X R0 $\Leftarrow$ raddr0 = 5 R1 $\Leftarrow$ raddr1 = X R2 $\Leftarrow$ raddr2 = X R3 $\Leftarrow$ raddr3 = X Cycle 1 W0 $\Leftarrow$ we0 = 1 $\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xDEADBEEF W1 $\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X R0 $\Leftarrow$ raddr0 = 5 R1 $\Leftarrow$ raddr1 = X R2 $\Leftarrow$ raddr2 = X R3 $\Leftarrow$ raddr3 = X	$\Rightarrow$ rdata0 = 0x10000005 R1 $\Rightarrow$ rdata1 = X R2 $\Rightarrow$ rdata2 = X R3 $\Rightarrow$ rdata3 = X Cycle 1 R0 $\Rightarrow$ rdata0 = 0xDEADBEEF R1 $\Rightarrow$ rdata1 = X R2 $\Rightarrow$ rdata2 = X R3 $\Rightarrow$ rdata3 = X
2.2	All read data is 0 and unable to write when active low reset signal is 0	Cycle 0 $\Leftarrow$ RESET = 0 W0 $\Leftarrow$ we0 = 1 $\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xDEADBEEF W1 $\Leftarrow$ we1 = 1 $\Leftarrow$ waddr1 = 12	Cycle 0 R0 $\Rightarrow$ rdata0 = 0x00000000 R1 $\Rightarrow$ rdata1 = 0x00000000 R2 $\Rightarrow$ rdata2 = 0x00000000 R3 $\Rightarrow$ rdata3 = 0x00000000

		↵ wdata1 = 0xCAFEBAE R0 ↵ raddr0 = 5 R1 ↵ raddr1 = 12 R2 ↵ raddr2 = 3 R3 ↵ raddr3 = 0 Cycle 1 ↵ RESET = 1 W0 ↵ we0 = 1 ↵ waddr0 = 5 ↵ wdata0 = 0xDEADBEEF W1 ↵ we1 = 1 ↵ waddr1 = 12 ↵ wdata1 = 0xCAFEBAE R0 ↵ raddr0 = 5 R1 ↵ raddr1 = 12 R2 ↵ raddr2 = 3 R3 ↵ raddr3 = 0	Cycle 1 R0 ⇒ rdata0 = 0xDEADBEEF R1 ⇒ rdata1 = 0xCAFEBAE R2 ⇒ rdata2 = 0x00000000 R3 ⇒ rdata3 = 0x00000000
--	--	---	---

Functionality 3: Edge Case Read/Write Functionality

Test Case	Description	Input Sequence	Expected Output
3.1	4 simultaneous reads to the same register	Begin in INITIALIZED STATE Cycle 0 W0 ↵ we0 = 0 ↵ waddr0 = X ↵ wdata0 = X W1	Cycle 0 R0 ⇒ rdata0 = 0x10000007 R1 ⇒ rdata1 = 0x10000007 R2

		$\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X R0 $\Leftarrow$ raddr0 = 7 R1 $\Leftarrow$ raddr1 = 7 R2 $\Leftarrow$ raddr2 = 7 R3 $\Leftarrow$ raddr3 = 7 Cycle 1 W0 $\Leftarrow$ we0 = 1 $\Leftarrow$ waddr0 = 7 $\Leftarrow$ wdata0 = 0xDEADBEEF W1 $\Leftarrow$ we1 = 0 $\Leftarrow$ waddr1 = X $\Leftarrow$ wdata1 = X R0 $\Leftarrow$ raddr0 = 7 R1 $\Leftarrow$ raddr1 = 7 R2 $\Leftarrow$ raddr2 = 7 R3 $\Leftarrow$ raddr3 = 7	$\Rightarrow$ rdata2 = 0x10000007 R3 $\Rightarrow$ rdata3 = 0x10000007 Cycle 1 R0 $\Rightarrow$ rdata0 = 0xDEADBEEF R1 $\Rightarrow$ rdata1 = 0xDEADBEEF R2 $\Rightarrow$ rdata2 = 0xDEADBEEF R3 $\Rightarrow$ rdata3 = 0xDEADBEEF
3.2	2 simultaneous writes to the same register with same data	W0 $\Leftarrow$ we0 = 1 $\Leftarrow$ waddr0 = 5 $\Leftarrow$ wdata0 = 0xCAFEBAFE W1 $\Leftarrow$ we1 = 1 $\Leftarrow$ waddr1 = 5 $\Leftarrow$ wdata1 = 0xCAFEBAFE R0 $\Leftarrow$ raddr0 = 5 R1 $\Leftarrow$ raddr1 = 5	R0 $\Rightarrow$ rdata0 = 0xCAFEBAFE R1 $\Rightarrow$ rdata1 = 0xCAFEBAFE R2 $\Rightarrow$ rdata2 = X R3 $\Rightarrow$ rdata3 = X

		R2 ⇨ raddr2 = X R3 ⇨ raddr3 = X	
3.3	2 simultaneous writes to the same register with different data	W0 ⇨ we0 = 1 ⇨ waddr0 = 5 ⇨ wdata0 = 0xDEADBEEF W1 ⇨ we1 = 1 ⇨ waddr1 = 5 ⇨ wdata1 = 0xCAFEBAFE R0 ⇨ raddr0 = 7 R1 ⇨ raddr1 = 7 R2 ⇨ raddr2 = X R3 ⇨ raddr3 = X	R0 ⇨ rdata0 = 0xDEADBEEF R1 ⇨ rdata1 = 0xDEADBEEF R2 ⇨ rdata2 = X R3 ⇨ rdata3 = X
3.4	Register 0 is a Zero Register and always stores 0	W0 ⇨ we0 = 1 ⇨ waddr0 = 0 ⇨ wdata0 = 0xDEADBEEF W1 ⇨ we1 = 1 ⇨ waddr1 = 0 ⇨ wdata1 = 0xCAFEBAFE R0 ⇨ raddr0 = 0 R1 ⇨ raddr1 = 0 R2 ⇨ raddr2 = 0	R0 ⇨ rdata0 = 0x00000000 R1 ⇨ rdata1 = 0x00000000 R2 ⇨ rdata2 = 0x00000000 R3 ⇨ rdata3 = 0x00000000

		R3 ⇐ raddr3 = 0	
3.5	Reads can be performed asynchronously but not writes	Begin in INITIALIZED STATE Cycle 0 (launch/leading posedge) W0 ⇐ we0 = 1 ⇐ waddr0 = 5 ⇐ wdata0 = 0xDEADBEEF	Cycle 0 (on negedge) R0 ⇐ rdata0 = 0x10000005 R1 ⇐ rdata1 = 0x1000000C R2 ⇐ rdata2 = 0x10000005 R3 ⇐ rdata3 = 0x1000000C
		W1 ⇐ we1 = 1 ⇐ waddr1 = 12 ⇐ wdata1 = 0xCAFEBAFE	
		R0 ⇐ raddr0 = 5 R1 ⇐ raddr1 = 12 R2 ⇐ raddr2 = 5 R3 ⇐ raddr3 = 12	Cycle 0 (capture/trailing posedge) R0 ⇐ rdata0 = 0xDEADBEEF R1 ⇐ rdata1 = 0xCAFEBAFE R2 ⇐ rdata2 = 0xDEADBEEF R3 ⇐ rdata3 = 0xCAFEBAFE

Assertions

Test Case	Description
4.1	The value of Register 0 must always remain 0
4.2	Active-low synchronous reset should set every register to 0
4.3	Read port outputs must reflect changes in register values within the same timestep
4.4	If we0 and we1 == 1 and waddr0 == waddr1, the register must take data from wdata0

Coverage

Case	Description	Coverage Items	Bins
5.1	Read Ports	Coverpoints - cp_raddr0 (raddr0) - cp_raddr1 (raddr1) - cp_raddr2 (raddr2)	32 bins for each read port: bins all_regs[] = {[0:31]}

	Every read port (R0, R1, R2, R3) hits every possible address	cp_raddr3 (raddr3)	
5.2	Write Ports Every write port (W0, W1) hits every possible address	Coverpoints - cp_waddr0 (waddr0) - cp_waddr1 (waddr1)	32 bins for each write port: bins all_regs[] = {[0:31]}
5.3	Write Enables Assert all combinations of the two write enables	Coverpoints - cp_we0 (we0) - cp_we1 (we1) Crosses - cross_write_enables (cp_we0, cp_we1)	Default bins created by cross: we0 x we1 {00, 01, 10, 11}
5.4	Write-After-Write (WAW) Both write ports simultaneously write to the same register	Crosses cross_write_collision (cp_waddr0, cp_waddr1, cp_we0, cp_we1)	Only the bins created by the cross that satisfy the condition: cp_waddr0 == cp_waddr1
5.5	Read-After-Write (RAW) Writing to a register and then reading from it simultaneously	Coverpoints This coverpoint is a ternary operator that results in: - Any address that is being written to and read from (waddr == raddr) in the same transaction - Or -1 if the no register is being written to and read from at the same time	One bin for each register: bins all_regs[] = { [0:31] } If the coverpoint is -1, no bin is hit
5.6	Concurrent Read All read ports access the same address simultaneously	Crosses cross_simultaneous_read (cp_raddr0, cp_raddr1, cp_raddr2, cp_raddr3)	Only the bins created by the cross that satisfy the condition: cp_raddr0 == cp_raddr1 && cp_raddr0 == cp_raddr2 && cp_raddr0 == cp_raddr3
5.7	Reset Reset was toggled	Coverpoint cp_reset (reset)	Two bins for both reset states {0, 1}